

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/228445903>

An hybrid genetic algorithm for two-dimensional cutting problems using guillotine cuts

Article · April 2012

CITATIONS

2

READS

219

2 authors:



[Hamza Gharsellaoui](#)

University of Carthage

82 PUBLICATIONS 242 CITATIONS

[SEE PROFILE](#)



[Hamadi Hasni](#)

Ecole Nationale des Sciences de l'Informatique

12 PUBLICATIONS 66 CITATIONS

[SEE PROFILE](#)

Some of the authors of this publication are also working on these related projects:



Safety Control and Optimization of Embedded Databases [View project](#)



Research [View project](#)

An hybrid genetic algorithm for two-dimensional cutting problems using guillotine cuts

Hamza Gharsellaoui¹, Hamadi Hasni²

¹National Institute of Applied Sciences and Technologies, INSAT, Tunisia
Gharsellaoui.hamza@gmail.com

²National School of Computer Science, ENSI, University of Manouba, Tunisia
hamadi.hasni@ensi.rnu.tn

ABSTRACT

The paper deals with the purpose of one hybrid approach for solving the constrained two-dimensional cutting (2DC) problem. We study this hybrid approach that combines the genetic algorithm and the Tabu search method. For this problem, we assume a packing of a whole number of rectangular pieces to cut, and that all cuts are of guillotine type in one sheet of a fixed width and an infinite height. Finally, we undertake an extensive experimental study with a large number of problem instances extracted from the literature by the Hopper's Benchmarks in order to support and to prove our approach and to evaluate the performance.

Key words: guillotine cutting and packing problem, Tabu search, genetic algorithm, guillotine constraint, Hybrid approach.

1 INTRODUCTION

The packing problems have been widely studied during the last three decades, as they are often faced in industry. The rectangular pieces packing problem, cutting also from rectangular board, is one particular case of this set of problems. The aim is often to achieve the minimum trim loss [13]. We had done some studies on packing problems in [14]. In this paper we propose a combination tabu search / genetic algorithms approach to construction of optimization algorithms for problems such as packing problems whose in-

volve constructing an arrangement of items that minimizes the total space required by the arrangement. This is mainly due to the constraints imposed by the industrial applications, e.g. textile, wood, steel and metal industry. A recent survey on packing problems is given in [15]. In this paper, we specifically consider the two-dimensional (2D) rectangular strip packing problem based on a new hybrid approach, named hybrid genetic algorithm. The input is a list of n rectangles with their dimensions (length and width). The goal is to pack the rectangles without overlap into a single rectangle of width W and minimum height H . We further restrict ourselves to the oriented, orthogonal variation, where rectangles must be placed parallel to the horizontal and vertical axes, and the rectangles can be rotated. Further, for our test cases, all dimensions are integers. Like most packing problems, 2D rectangular strip packing (even with these restrictions) is NP-hard. Finally, our algorithm naturally solves a more general problem: given a set of rectangles and a target rectangle, find a packing of a subset of those rectangles which gives an optimal packing of the target. Numerical examples also showed the superiority of the proposed algorithm compared with two classical methods in the literature (pure genetic algorithm and hopper's results). This paper is organized as follows. In Section 2, we provide the problem description and the literature review. In Section 3, we present the resolution methods which uses the bottom left algorithm and the guillotine constraint. In Section 4, we show how our hybrid algorithm can be adapted for solving the general 2DC problem. In Section 5, we

undertake a comparative study of our proposed algorithm and evaluate its performance for the 2DC problem using benchmark problems from the literature. Finally, in Section 6, we summarize the contributions of this paper and explain its possible extensions.

2 PROBLEM DESCRIPTION AND LITERATURE REVIEW

2.1 Definitions

The goal of our research problem is to minimize the waste and the total used space in a set of rectangles and a target rectangle, e.g find a packing of a subset of those rectangles which gives an optimal packing of the target. We have found in our related work on packing problems that cutting and packing (CP) problems have numerous industrial applications, spanning from the direct use of CP models (production, loading cargo into ships, vehicles, containers, wood) to their abstract adaptation to more complex problems (scheduling problems, budgeting, etc). A significant improvement of the proposed algorithm results, over those obtained by either the Hopper results and pure genetic algorithm (GA), has been achieved.

2.1.1 Genetic Algorithm (GA)

A genetic algorithm (GA) is a procedure used to find approximate solutions to search problems through application of the principles of evolutionary biology. Genetic algorithms use biologically inspired techniques such as genetic inheritance, natural selection, mutation, and sexual reproduction (recombination, or crossover). For this problem, members of a space of candidate solutions, called individuals, are represented using abstract representations called chromosomes. The GA consists of an iterative process that evolves a working set of individuals called a population toward an objective function, or fitness function. The evolutionary process of a GA is a highly simplified and stylized simulation of the biological version. It starts from a population of individuals randomly generated according to some probability distribution, usually uniform and updates

this population in steps called generations. Each generation, multiple individuals are randomly selected from the current population based upon some application of fitness, bred using crossover, and modified through mutation to form a new population [1].

2.1.2 Tabu Search (TS) Algorithm

Tabu search is a metaheuristic that guides local heuristic search procedures to explore the solution space beyond local optimality. It was introduced by Glover [2, 3] specifically for combinatorial problems. Since then, tabu search has successfully been applied to a wide range of problems. For example, tabu search has been applied to flow shop scheduling [4, 5], architectural design [6], time tabling problem [7], among others. The tabu search starts at some initial point and then moves successively among neighboring points. At each iteration, a move is made to the best point in the neighborhood of the current point which may not be an improving solution. The method forbids (makes tabu) points with certain attributes with the goals of preventing cycling and guiding the search towards unexplored regions of the solution space. This is done using an important feature of the tabu search method called tabu list. A tabu list consists of the latest moves made so that recently visited points are not generated again. The size of the tabu list can be either fixed or variable.

2.2 Literature Review

Further background Packing problems in general are important in manufacturing settings; for example, one might need n specific rectangular pieces of glass to put together a certain piece of furniture, and the goal is to cut those pieces from the minimum height fixed-width piece of glass. The more general version of the problem allows for irregular shapes, which is required for certain manufacturing problems such as clothing production. However, the rectangular case has many industrial applications [8]. The 2D rectangular strip packing problem has been the subject of a great deal of research, both by the theory community and the operations-research community [9,10]. One focus has been on heuristics that

lead to good solutions in practice. One line in this area considers simple heuristics for greedily placing an ordered list of rectangles, the most widely used and well-studied of which is the Bottom-Left heuristic.

3 RESOLUTION METHODS

In this section, we explain the techniques used to resolve cutting and packing (CP) problems. Because the Bottom-Left heuristic is a foundation for our work, we describe it in some details.

3.1 Bottom Left (BL) Algorithm

The Bottom-Left (BL) heuristic was introduced in [11]. The placement Bottom Left (BL) algorithm using the permutation is executed to place rectangular pieces into the main sheet (object). We can describe the BL algorithm process using the following definition: This Bottom-Left placement algorithm takes a sheet size and an input sequence of pieces and their allowable rotations (rotation criteria). The algorithm progresses packing by placing the first piece in the lower left corner of the sheet in its most efficient orientation (the orientation that yields the smallest bounding rectangle height within the set of rotation criteria). With subsequent pieces, a valid location for placement is found by testing for intersections and containment. If the piece is not intersecting by (or containing) other already placed pieces, then the location of the piece is valid and therefore can be assigned to the sheet. When a piece is in a position that intersects with already assigned pieces, we use the rotation technique. The process continues as before with overlap/intersection tests and resolution until the piece does not intersect and can be placed. Packing is completed when all pieces have been assigned to the sheet.

3.2 Guillotine Cutting Problems

The two-dimensional guillotine-cutting problem has been widely studied in the operational research literature. The unrestricted problem is known to be NP-hard. When cutting specific materials like glass it may be required that the rectangles can be cut out of the bin by a number of

guillotine cuts which can be thought of as edge-to-edge cuts. The most common constraint requires guillotine cutting patterns; i.e., patterns where pieces can be obtained using a series of horizontal and vertical cuts. This constraint demands that all placed pieces are reproducible through a series of guillotine cuts. As is known, a guillotine cut through a rectangle runs from one edge to the opposite edge and parallel to the other two edges of the rectangle in a straight line. That is, there should exist a series of face parallel straight cuts that can cut the bin into pieces so that each piece contains a box and no box has been intersected by a cut. In our example, the first packing is guillotine cuttable and the solution to obtain a guillotine cut is feasible (Figure 1),

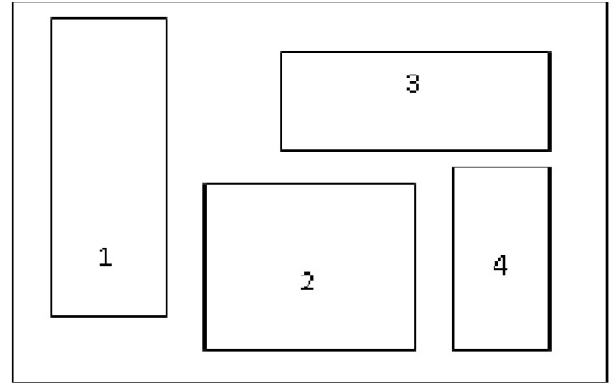


Figure 1: Guillotine Configuration

while the second packing is not and the solution to obtain a guillotine cut is unfeasible (Figure 2).

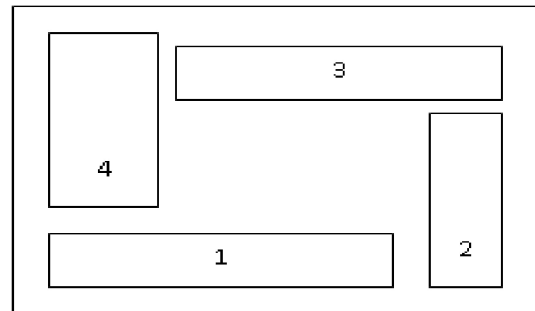


Figure 2: Non-Guillotine Configuration

Notation and example:

Alternatively, a packing pattern (sequence) can be represented by a permutation π [12].

i is the index of the rectangle (r_i).

$\pi = (i_1, \dots, i_n)$ is a permutation.

The permutation represents the sequence in which the pieces (rectangles) are packed. The advantage of this data structure is the facile creation of new permutations by changing the sequence. A consequence of the variable data structure is the fact that every permutation has to be assigned to a unique packing pattern.

Allocation of some of pieces with BL is illustrated in Figure 3. As can be seen in Figure 3,

In $\pi_1 = (-1, 3, -2, 4)$

piece 1 is placed first to the bottom and then to the left as far as possible with a rotation of 90° , (the signe - represents a rotation of the piece by 90°). Pieces 3 and 2 are placed in the same manner with a rotation of the piece 2. Then, piece 4 is placed in its optimal location.

In $\pi_2 = (-1, 2, 3, 4)$

piece 1 is placed first to the bottom and then to the left as far as possible with a rotation. Piece 2 is then placed in its optimal location and pieces 3 and 4 are placed after that.

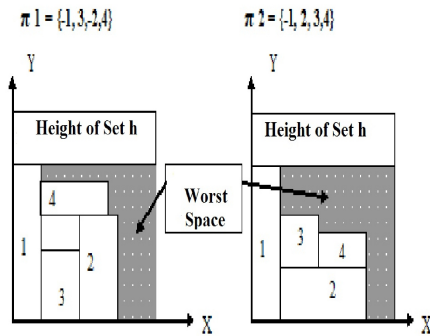


Figure 3: 2 possible permutations of pieces (1, 2, 3, 4).

In addition, the cost of the BL-algorithm is $\Theta(n^2)$. This based on the fact, that each rectangle (piece) r_i can be shifted a maximum of i times, because each shift is limited by one of the $i - 1$ placed rec-

tangles (pieces) or by the corners of the board. Hence, the cost of placing rectangle r_i is $\Theta(i)$ and the whole cost amounts to $\Theta(n^2)$.

4 HYBRID APPROACH DESCRIPTION

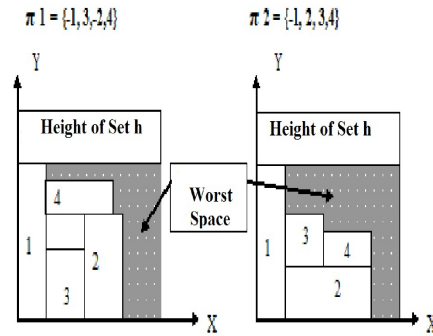
A genetic search algorithm is a heuristic search process that resembles natural selection. There are many variations and refinements, but any genetic algorithm has the features of reproduction, crossover and mutation. Initially a population is selected, and by means of crossovers among members of the population or mutation of members, the better of the population will remain. In the case of our hybrid approach the mutation is replaced by the Tabu Search (TS) Algorithm. For this reason we call this genetic method as an hybrid genetic approach.

4.1 Fitness-Function

For the genetic algorithm, the evaluation of a model set is obligatory, this is represented by a Fitness-Function $f: \pi \rightarrow R_+$ with the propriety $f(\pi_i) > f(\pi_j)$ if π_i is better than π_j .

The Fitness-Function value is inversely proportional to the height of a model set: $f(\pi) = 1/h(\pi)$; Where $h(\pi)$ is the model set height follow to the permutation π created by the BL algorithm.

π represents one permutation (arrangement) of rectangular pieces. The following picture (figure 3) is an example of placement(arrangement) of 4 pieces (1, 2, 3, 4).



To reducing the internal waste we have developed two variants of the placement (arrangement) as shown in the figure 3 with the same height of the rectangle (sheet) but different in quality, so by logic they have not the same Fitness Function. With the last definition of the Fitness Function ($f(\pi) = 1/h(\pi)$) if two set placement have the same height, their Fitness Function are equal even if one is better than the other. For this reason, it's necessary to define another Fitness Function more better. Following the last picture (figure 3), it's clear that π_2 is better than π_1 because the remaining space in the result configuration is better with π_2 . With this comparison, and if we consider only one objective which is the wastage minimization. Our Fitness Function is given by the following formula:

$$F(\pi) = H(\pi) + \text{Area (worst remaining space)} / h(\pi)^* \text{ width}$$

Where:

- $H(\pi) = H - h(\pi)$, where H is a maximum given height that guarantee $H(\pi)$ be a positive value.
- $h(\pi)$ is the model set height follow to the permutation π .
- Area (worst remaining space) is the area of the remaining space in the placement (arrangement) model following to the permutation π .
- width is a target (rectangle) width dimension.
- $h(\pi)^* \text{ width}$ is the rectangle area occupied by the placed rectangles (items).

4.2 Initialisation

it's evident that genetic algorithms uses m objects. Here the m objects are m permutations: $\pi_1, \pi_2, \dots, \pi_m$. We assign for each arrangement it's Fitness Function value: $f_i = f(\pi_i); i = 1, 2, \dots, m$. We consider the permutation π_i and it's Fitness Function f_i together like one individu $A_i: A_i = (\pi_i, f_i)$.

4.3 Recombination

The crossover operator is special in this case in order to respect the constraint that every piece

which is copied didn't repeated another time in order to respect the pieces unicity and creates two solutions or childs (permutations) $\pi_{i\text{new}}$ and $\pi_{j\text{new}}$ by combining two parents π_i and π_j . For each pair of parents (permutations), two crossover integers P_1 and P_2 are randomly chosen (generated) with the condition $1 \leq P_1, P_2 \leq n$. In the random position P_1 , the crossover operator copy P_2 elements from π_i for the begining of $\pi_{i\text{new}}$ and copy P_2 elements from π_j for the begining of $\pi_{j\text{new}}$. Then, $\pi_{i\text{new}}$ is completed by the remaining elements of π_j with respect to the same order of appearance and $\pi_{j\text{new}}$ is completed also by the remaining elements of π_i with respect also to the same order of appearance.

Example

$\pi_i = (1, -2, 3, 4, -5, 6)$ and $\pi_j = (-6, 4, 2, 5, -3, 1)$

if ($P_1 = 2$ and $P_2 = 3$) then

$\pi_{i\text{new}}(1) = \pi_i(P_1) = \pi_i(2) = -2;$

$\pi_{i\text{new}}(2) = \pi_i(P_{1+1}) = \pi_i(3) = 3;$

$\pi_{i\text{new}}(3) = \pi_i(P_{1+2}) = 4;$

$\pi_{i\text{new}}(4) = \pi_j(1) = -6;$

$\pi_{i\text{new}}(5) = \pi_j(4) = 5;$

$\pi_{i\text{new}}(6) = \pi_j(6) = 1;$

Then, $\pi_i \text{ new} = (-2, 3, 4, -6, 5, 1)$. With the same method, $\pi_j \text{ new} = (4, 2, 5, 1, 3, 6)$. So, with the crossover processus we get m new permutations.

4.4 Tabu Search

This algorithm was proposed to the cutting problem including the guillotine constraint and replaced the mutation operator and was evaluated in a series of numerical experiments that are run on problem instances taken from the literature, as well as on randomly generated instances which proved our approach. For this case, we consider the model set X composed by n permutations (arrangements) which are represented by a rectangular items (pieces). We adopt the change of one piece by another of their neighbors as an elementary transformation and we evaluate the Fitness-Function f for each arrangement set.

– The neighborhood function

Let $T_{i,j}, (when(i,j) \in [1..n] * [1..n]; i \neq j)$; the transformation that consists of the changement

of the piece i by the piece j . The neighborhood of the current solution π_c , that corresponds to all possible transformations is $N(\pi_c)$. for each taboo iteration, the current solution π_c must be updated by changing it by the best solution of it's neighborhood that minimizes the Fitness-Function f . This piece of the best solution was added to the taboo List.

4.5 The Tournament Selection

The principle of selection by tournament increases the chances for poor individuals (arrangements) to participate in the improvement of the population. The principle is very quick to implement. A tournament consists of a meeting between two arrangements (parents) at random in the population. The tournament winner is the arrangement better P_1 (parent P_1). We repeat this method to select the second best arrangement P_2 (parent P_2). The principle of the arrangement (parent) P selection is the following:

1. $(P_1, P_2) = \text{random}(|n|)$.
2. $P = \text{random}(P_1, P_2)$.

5 EXPERIMENTATION AND ANALYSIS OF RESULTS

The algorithm was tested against the Hopper results and pure genetic algorithm (classic) GA. The results indicate that performance of the combined (hybrid) algorithm is better than that of either Hopper results or pure (classic) GA. We now present experimental results demonstrating the effectiveness of our method for finding perfect packing set. We use the benchmarks developed by Hopper. Those benchmarks contain collections with size ranging from 17 to 199 rectangles (items), all the instances are tested into a single rectangle of width ($W = 300$) and minimum height ($H = 250$). We evaluate our hybrid algorithm on the guillotinable instances from this set by an interesting comparison between the running results which are summarized in Table 1. This table reports, for each collection of instances (files) the number of items or placed rectangles (n), and the average of occupied space of both the three methods (pure genetic algorithms (Classic (%)), Hopper results (Hopper (%)) and

our hybrid genetic approach (Hybrid (%)).

Algorithm		Classic (%)	Hybrid (%)			Hopper (%)
Data ^[16]	N	✓	Low	Average	Better	✓
t1c.xls	17	88.22	88.60	91.62	92.67	83.47
t2a.xls	25	91.98	93.07	93.73	93.95	95.73
t2b.xls	25	91.78	92.16	92.61	93.10	88.16
t2c.xls	25	88.33	92.17	92.17	92.17	83.68
t3a.xls	29	88.11	91.32	93.46	93.90	90.91
t3b.xls	29	86.58	91.74	92.94	93.46	89.68
t3c.xls	29	90.50	93.02	93.15	93.46	88.46
t4a.xls	49	93.46	93.89	94.38	95.24	89.68
t4b.xls	49	91.74	94.78	95.10	95.24	89.68
t4c.xls	49	91.11	96.15	96.15	96.15	88.49
t5a.xls	73	93.46	95.78	95.14	95.24	93.89
t5b.xls	73	93.88	95.69	95.92	96.15	89.28
t5c.xls	73	92.17	94.33	95.14	95.24	88.10
t6a.xls	97	94.48	96.75	96.75	96.75	94.48
t6b.xls	97	94.79	95.23	95.64	96.15	93.89
t6c.xls	97	94.34	95.24	95.92	96.15	95.69
t7a.xls	199	95.56	97.09	97.09	97.09	95.23
t7b.xls	199	95.69	95.69	96.36	96.93	96.15
t7c.xls	199	95.71	96.16	96.69	97.14	93.90

Table 1: Comparison results applied to the Hopper Benchmarks

By Examining Table 1 and considering only those instances, we prove the performance of our hybrid algorithm and we observe that:

1. The Hybrid Genetic Algorithm (Hybrid (%)) performs better than pure genetic algorithms (Classic (%));
2. The Hybrid Genetic Algorithm (Hybrid (%)) performs better than Hopper results (Hopper (%)), and offer more gains in terms of occupied space, and minimum waste of sheet.

6 CONCLUSION AND FUTURE TRENDS

In this paper we have proposed an hybrid approach for solving the constrained two-dimensional cutting (2DC) problem. Starting from a packing with height H , this approach tries to solve the (2DC) problem with decreasing values of H while avoiding solutions with

tall and thin wasted spaces. A specific fitness function f is designed to guide the search. Computational results show that our approach is very promising. The approach was tested and the results indicate that performance of the combined algorithm is better than others. Since we were unable to find a counterexample for which the approach fails, we conjecture that it always finds an optimal constrained guillotine cutting. Numerical examples also showed the superiority of the proposed approach compared with the classical methods in the literature.

There are some practical issues that need to be addressed. However, this hybrid approach optimises the solution to minimize the chutes and the waste. This should be extended to include other parameters such as considering three-dimensional cutting (3DC) problem. Further research in this area will include studying the impact of both rectangular and irregular forms of pieces on guillotine cutting problems.

REFERENCES

- [1] J. E. Beasley: Algorithms for unconstrained two-dimensional guillotine cutting. *Journal of the Operational Research Society*, 1985, 36, 297-306
- [2] Glover, F., 1989, "Tabu Search-Part I", *ORSA Journal on Computing*, 1, pp. 190-206.
- [3] Glover, F., 1990, "Tabu Search-Part II", *ORSA Journal on Computing*, 2, pp. 4-32.
- [4] Widmer, M., and A. Hertz, 1989, "A new heuristic method for the ow shop sequencing problem", *European Journal of Operational Research*, 41, pp. 186-193.
- [5] Taillard, E., 1990, "Some efficient heuristic methods for the ow shop sequencing problem", *European Journal of Operational Research*, 47, pp. 65-74.
- [6] Bland, J.A., G.P. Dawson, 1991, "Tabu search and design optimization", *Computer-aided Design*, 23, 3, pp. 195-201.
- [7] Hertz, A., 1991, "Tabu search for large scale time tabling problems", *European Journal of Operational Research*, 51, pp. 39-47.
- [8] Hifi- Mhand. The DH/KD algorithm: a hybrid approach for unconstrained two-dimensional cutting problems. *European Journal of Operational Research* (1997) 97, 41-52.
- [9] Hifi- Mhand, Zissimopoulos V. A recursive exact algorithm for weighted two-dimensional cutting. *European Journal of Operational Research* (1996) 91, 55-64.
- [10] Morabito R, Arenales M. An and/or graph approach to the container loading problem. *International Transactions in Operational Research* (1994) 58, 63-71.
- [11] Gilmore PC, Gomory RE. The theory and computation of knapsack functions. *Operations Research* (1966), 1045-1074.
- [12] S. Jakobs: On genetic algorithms for packing polygons, *Euro. J. of Op. Res.*, 88 (1996) 165-181.
- [13] H.F. Teng, D. Liu, An improved BL-algorithm for genetic algorithm of the orthogonal packing of rectangles, *European Journal of Operational Research* 112 (1999) 413-420.
- [14] H.F. Teng, Z.C. Yu, X. Gao, Layout optimization for the dishes installed on a rotating table, *Science in China (series A)* (1994) 1272-1280.
- [15] K.A. Dowsland, W.B. Dowsland, Packing problems, *European Journal of Operational Research* 56 (1992) 2-14.
- [16] Hopper E & Turton BCH, (2002), Problem Generators for Rectangular Packing Problems, *Studia Informatica Universalis*, 2(1), 123-136.